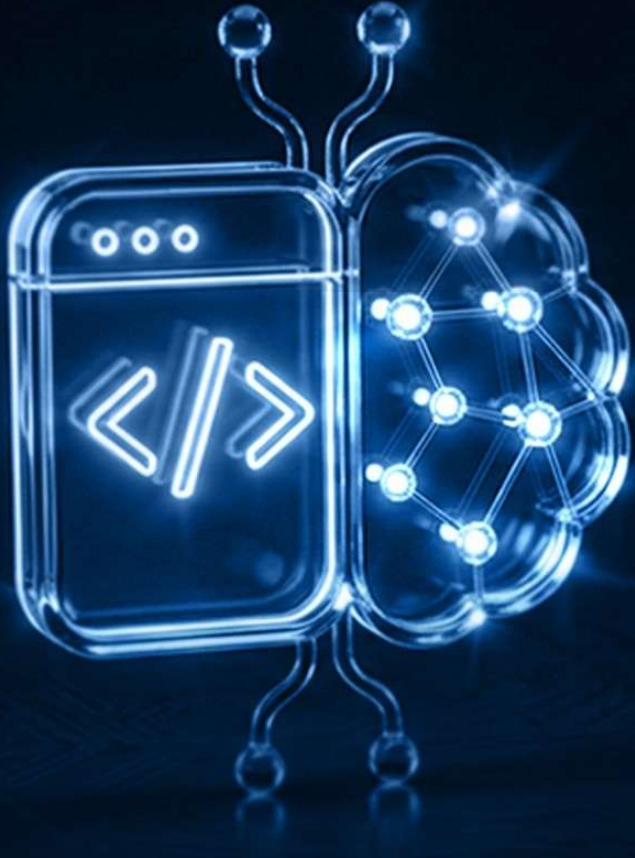


دوره آموزشی

GenAI

مهندسی نرم افزار در عصر هوش مصنوعی

مدرس: علیرضا ارومند



جلسه سوم

هوش مصنوعی واقعاً چیست و مدل چگونه یاد می‌گیرد؟

دوره آموزشی

GenAI

مهندسی نرم افزار در عصر هوش مصنوعی

مدرس: علیرضا ارومند





بخش اول —
مدل چیست و چرا
یادگیری ماشین
متفاوت است؟



قرار است امروز چه چیزی را بفهمیم؟



- مدل واقعاً چیست؟
- یادگیری دقیقاً کجا اتفاق می‌افتد؟
- متن چگونه برای ماشین قابل پردازش می‌شود؟
- معنا چگونه در مدل شکل می‌گیرد؟
- این مفاهیم چه ربطی به معماری دارند؟



برنامه‌نویسی سنتی و یادگیری ماشین

- در برنامه‌نویسی، قوانین را خودمان می‌نویسیم
- در یادگیری ماشین، مدل قوانین را می‌آموزد
- داده جایگزین منطق برنامه نمی‌شود
- مسئله، روش حل را تعیین می‌کند



الگوریتم، معماری و مدل یکی نیستند



- الگوریتم، روش انجام کار است
- معماری، ساختار محاسبات را تعیین می‌کند
- مدل، نتیجه فرایند یادگیری است
- مدل حاوی پارامترهای آموخته‌شده است

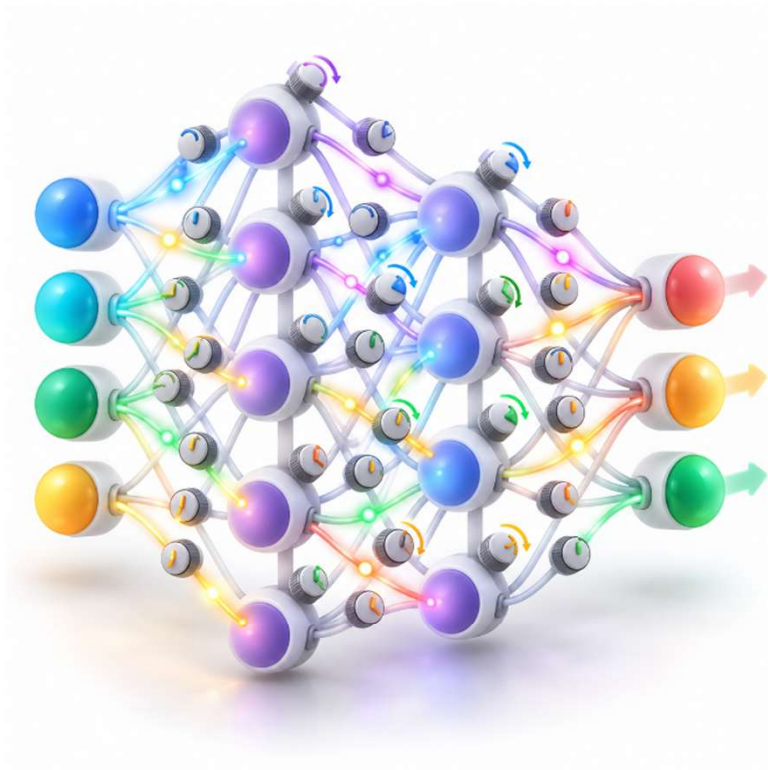


مدل واقعاً چیست؟

- مدل یک سیستم پارامتری است
- ورودی را به خروجی نگاشت می‌کند
- رفتارش از آموزش شکل گرفته است
- مدل مجموعه‌ای از قوانین دستی نیست



پارامترها؛ چیزی که واقعاً یاد گرفته می‌شود

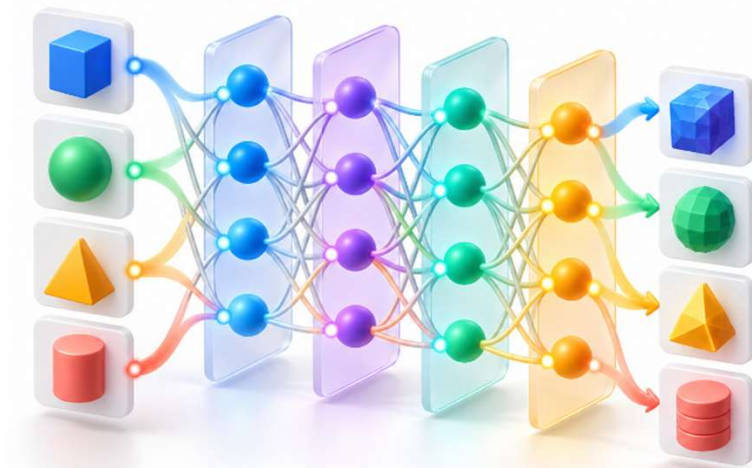


- پارامترها رفتار مدل را تعیین می‌کنند
- وزن‌ها نوعی پارامتر هستند
- Bias نیز یک پارامتر قابل یادگیری است
- آموزش، مقادیر پارامترها را تغییر می‌دهد



شبکه عصبی چه چیزی به ما می‌دهد؟

- شبکه از لایه‌های محاسباتی ساخته می‌شود
- هر لایه ورودی را تبدیل می‌کند
- خروجی هر لایه وارد لایه بعدی می‌شود
- پارامترها نحوه این تبدیل را تعیین می‌کنند



آموزش و استنتاج دو فرایند متفاوت‌اند



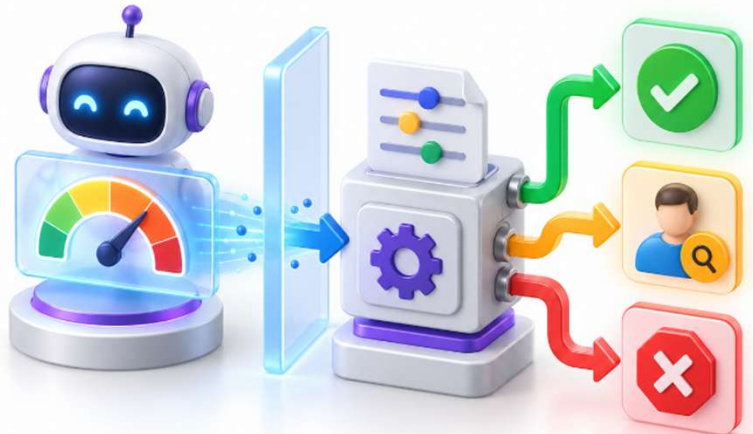
- Training یعنی تغییر دادن مدل
- Inference یعنی استفاده از مدل
- در استنتاج پارامترها معمولاً ثابت‌اند
- آموزش بسیار پرهزینه‌تر از استنتاج است

پیش‌بینی، حقیقت نیست

- مدل یک Prediction تولید می‌کند
- پیش‌بینی ممکن است اشتباه باشد
- Ground Truth از دنیای واقعی می‌آید
- این دو مفهوم نباید مخلوط شوند



پیش‌بینی، تصمیم کسب‌وکار هم نیست



- مدل احتمال یا امتیاز تولید می‌کند
- سیستم درباره اقدام تصمیم می‌گیرد
- سیاست کسب‌وکار مستقل از مدل است
- مرز این دو باید معماری شود



مدل فقط یک جزء از سیستم است

- مدل نباید مالک کل تصمیم باشد
- منطق قطعی همچنان جایگاه خودش را دارد
- داده معتبر بیرون مدل زندگی می‌کند
- گاهی انسان باید تصمیم نهایی بگیرد



بخش دوم —
مدل چگونه یاد می‌گیرد؟



قلب آموزش مدل

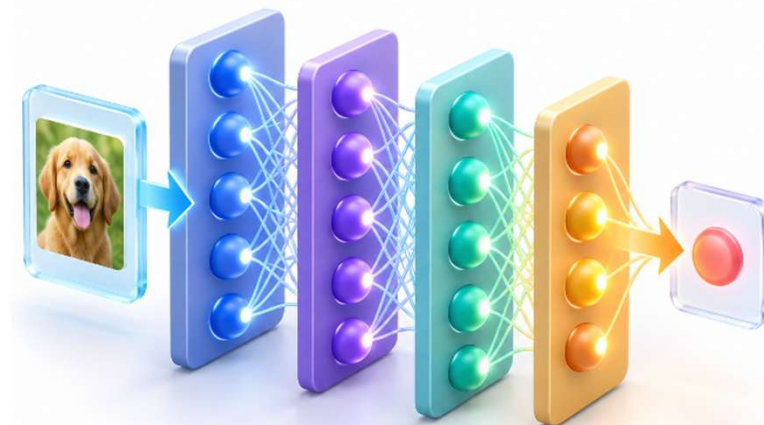


- مدل ابتدا پیش‌بینی می‌کند
- خطای پیش‌بینی محاسبه می‌شود
- منشأ خطا پیدا می‌شود
- پارامترها در جهت بهتر تغییر می‌کنند
- این چرخه بارها تکرار می‌شود



گذر رو به جلو

- ورودی وارد مدل می‌شود
- محاسبات از لایه‌ها عبور می‌کنند
- پارامترهای فعلی استفاده می‌شوند
- در انتها Prediction تولید می‌شود



مدل از کجا می‌فهمد اشتباه کرده؟



- پیش‌بینی به‌تنهایی کافی نیست
- به یک مرجع برای مقایسه نیاز داریم
- Ground Truth پاسخ مورد انتظار است
- اختلاف باید قابل اندازه‌گیری شود

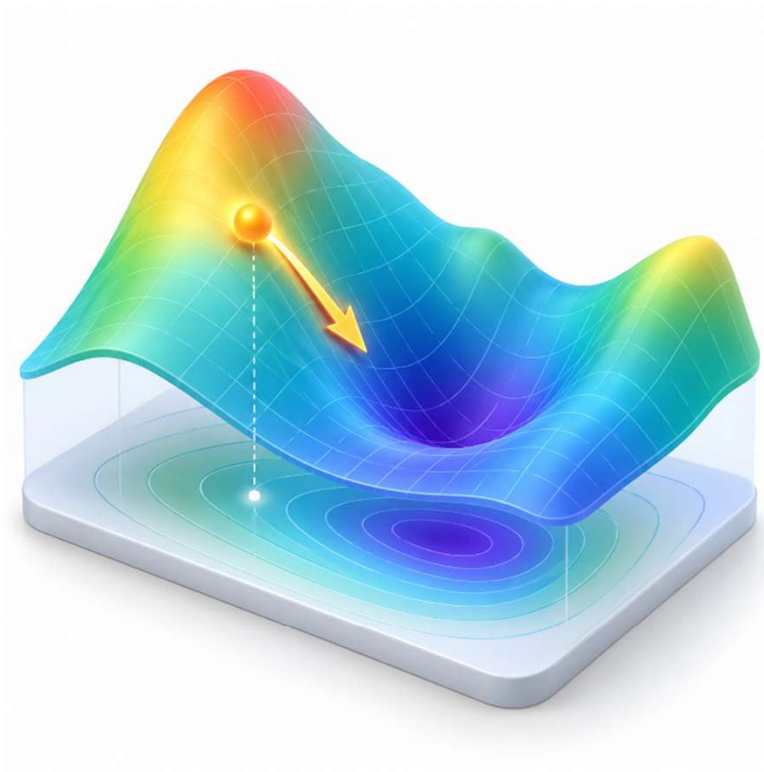


تابع زیان؛ خطا را به عدد تبدیل می‌کنیم

- Loss میزان بدی پیش‌بینی را اندازه می‌گیرد
- Loss کوچک‌تر معمولاً بهتر است
- نوع مسئله، Loss مناسب را تعیین می‌کند
- آموزش تلاش می‌کند Loss کاهش یابد



گرادیان، خود خطا نیست

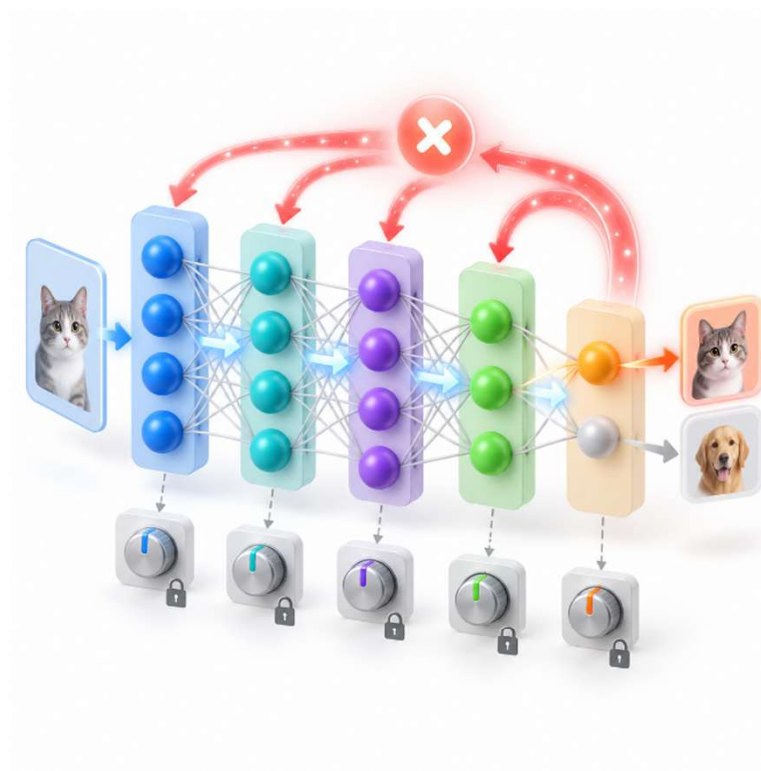


- Gradient مقدار Loss نیست
- حساسیت Loss به پارامترها را نشان می‌دهد
- جهت مناسب تغییر را مشخص می‌کند
- اندازه تغییر هنوز تصمیم‌گیری نشده است

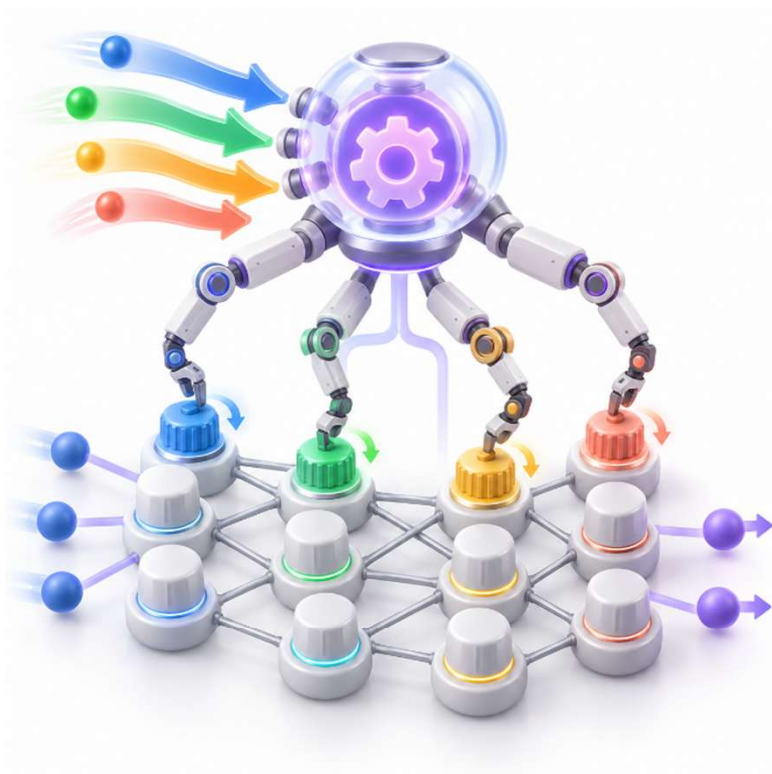


پس انتشار؛ خطا را به عقب برمی گردانیم

- محاسبه از خروجی به عقب انجام می شود
- سهم پارامترها در خطا بررسی می شود
- برای پارامترها Gradient محاسبه می شود
- هنوز هیچ پارامتری تغییر نکرده است



Optimizer؛ حالا پارامترها تغییر می‌کنند



- Gradient فقط اطلاعات تغییر را می‌دهد
- Optimizer تغییر واقعی را اعمال می‌کند
- هدف، کاهش Loss آینده است
- این فرایند بارها تکرار می‌شود



Learning Rate؛ اندازه قدم اهمیت دارد

- قدم خیلی بزرگ می‌تواند خطرناک باشد
- قدم خیلی کوچک آموزش را کند می‌کند
- Learning Rate اندازه حرکت را کنترل می‌کند
- مسیر درست بدون گام مناسب کافی نیست



Batch، Step و Epoch را قاطی نکنیم



- Batch بخشی از داده آموزشی است
- Step یک بار به روزرسانی مدل است
- هر Step معمولاً یک Batch می بیند
- Epoch یعنی عبور از کل داده



Checkpoint؛ مدل در میانه آموزش

- آموزش ممکن است بسیار طولانی باشد
- وضعیت مدل باید قابل ذخیره باشد
- Checkpoint امکان ادامه آموزش می‌دهد
- مدل میانی الزاماً مدل نهایی نیست



یادگیری از تکرار ساخته می‌شود



- یک Step مدل را باهوش نمی‌کند
- میلیون‌ها تغییر کوچک روی هم جمع می‌شوند
- Representation ها به تدریج شکل می‌گیرند
- رفتار نهایی حاصل کل مسیر است





بخش سوم —
مدل خوب فقط داده
آموزشی را حفظ نمی‌کند



هدف واقعی، تعمیم است

- مدل فقط برای داده آموزش ساخته نمی‌شود
- باید روی داده جدید هم خوب عمل کند
- این توانایی Generalization نام دارد
- حفظ کردن، یادگیری واقعی نیست



بیش‌برازش؛ وقتی مدل زیادی حفظ می‌کند

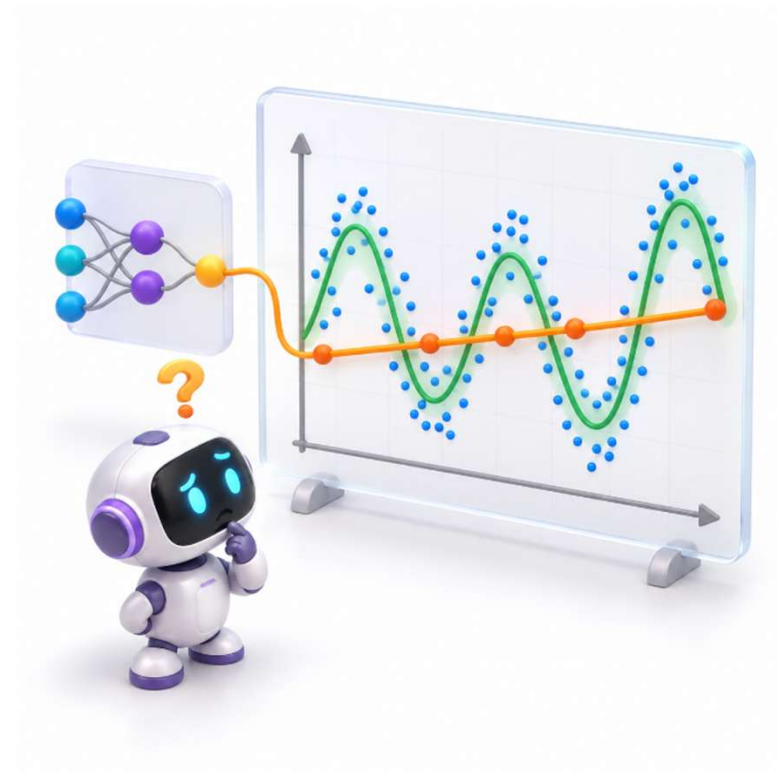


- Training Performance ممکن است عالی باشد
- داده جدید عملکرد ضعیفی نشان می‌دهد
- مدل الگوهای غیرضروری را آموخته است
- Loss پایین همیشه خبر خوبی نیست



کم‌برازش؛ مدل هنوز مسئله را نگرفته است

- مدل حتی آموزش را خوب یاد نگرفته
- ظرفیت یا آموزش کافی نیست
- الگوی اصلی هنوز استخراج نشده است
- Test و Training هر دو ضعیف‌اند



ارزیابی فقط نگاه کردن به Training نیست



- داده آموزش برای یادگیری استفاده شده
- ارزیابی باید روی داده ندیده انجام شود
- معیار مناسب به مسئله وابسته است
- کیفیت مدل چندبعدی است



همه مدل‌ها یک جور یاد نمی‌گیرند

- Supervised Learning پاسخ مرجع دارد
- Unsupervised Learning برچسب آماده ندارد
- Self-Supervised از خود داده هدف می‌سازد
- Reinforcement Learning بر پاداش متکی است



بخش چهارم —
وقتی مسئله ما
زبان است



حالا ورودی ما متن است

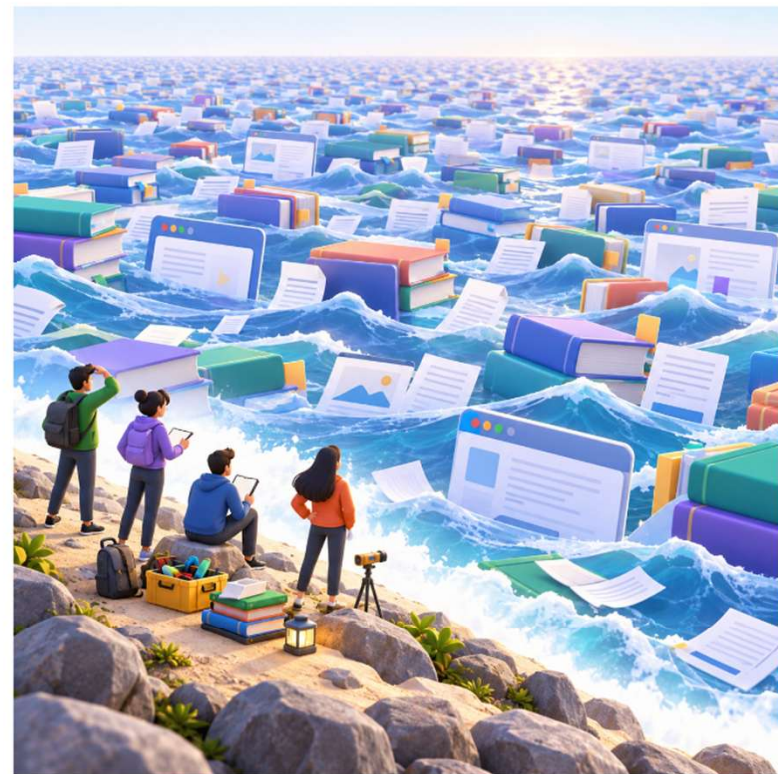


- شبکه عصبی با متن خام کار نمی‌کند
- متن ابتدا باید بازنمایی عددی شود
- اما فقط عددی کردن کافی نیست
- مدل باید ساختار زبان را نیز بیاموزد



برچسب‌گذاری اینترنت عملاً ممکن نیست

- حجم متن آموزشی بسیار بزرگ است
- انسان نمی‌تواند همه متن‌ها را برچسب بزند
- خود متن می‌تواند سیگنال آموزشی بسازد
- اینجا Self-Supervised Learning مهم می‌شود



مدل از خود متن سؤال می‌سازد



- بخشی از متن نقش ورودی می‌گیرد
- بخش دیگر پاسخ مورد انتظار می‌شود
- نیازی به برچسب‌گذاری دستی نیست
- داده خام تبدیل به داده آموزشی می‌شود



Training Objective رفتار مدل را جهت می‌دهد

- مدل باید بداند چه چیزی را بهینه کند
- Objective مسئله آموزشی را تعریف می‌کند
- در مدل زبانی، پیش‌بینی اهمیت دارد
- یادگیری حول همین هدف شکل می‌گیرد



Pre-training؛ یادگیری گسترده پیش از کاربرد خاص



- مدل حجم بزرگی از متن می‌بیند
- روابط زبانی به تدریج آموخته می‌شوند
- دانش عمومی در پارامترها شکل می‌گیرد
- کاربرد سازمانی مرحله دیگری است



مدل زبان دیتابیس حقایق نمی‌سازد

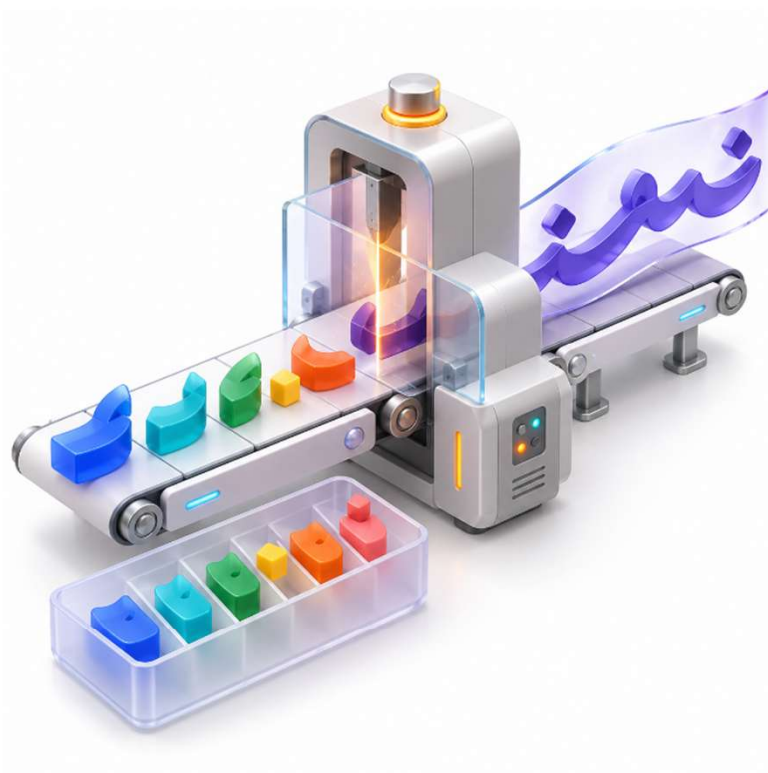
- مدل روابط آماری را یاد می‌گیرد
- Facts به شکل رکورد ذخیره نمی‌شوند
- پاسخ حاصل فرایند تولید است
- دانسته مدل با پایگاه داده متفاوت است



بخش پنجم –
متن چگونه وارد
مدل می‌شود؟



اولین مسئله: متن را چگونه بشکنیم؟



- مدل دنباله‌ای از واحدها را پردازش می‌کند
- این واحدها Token نام دارند
- Token الزاماً یک کلمه نیست
- Tokenizer نحوه شکستن را تعیین می‌کند



Token می‌تواند چه چیزی باشد؟

- گاهی یک کلمه کامل است
- گاهی بخشی از یک کلمه است
- نشانه‌ها نیز می‌توانند Token باشند
- زبان روی Token ها مدل می‌شود



چرا معمولاً از زیرواژه استفاده می‌کنیم؟



- روش حرف‌محور دنباله را طولانی می‌کند
- روش کلمه‌محور Vocabulary را بزرگ می‌کند
- Subword بین این دو تعادل ایجاد می‌کند
- کلمات جدید هم قابل بازنمایی می‌شوند



Vocabulary و طول دنباله با هم معامله می‌شوند

- Vocabulary بزرگ، Token های کمتری می‌خواهد
- Vocabulary کوچک، دنباله را طولانی می‌کند
- طول بیشتر یعنی پردازش بیشتر
- Tokenization روی هزینه اثر می‌گذارد



Tokenizer بخشی از قرارداد مدل است



- Tokenizer مستقل از مدل انتخاب نمی‌شود
- Vocabulary هنگام آموزش مشخص شده است
- تغییر Tokenizer رفتار ورودی را عوض می‌کند
- تعداد Token روی Context اثر دارد

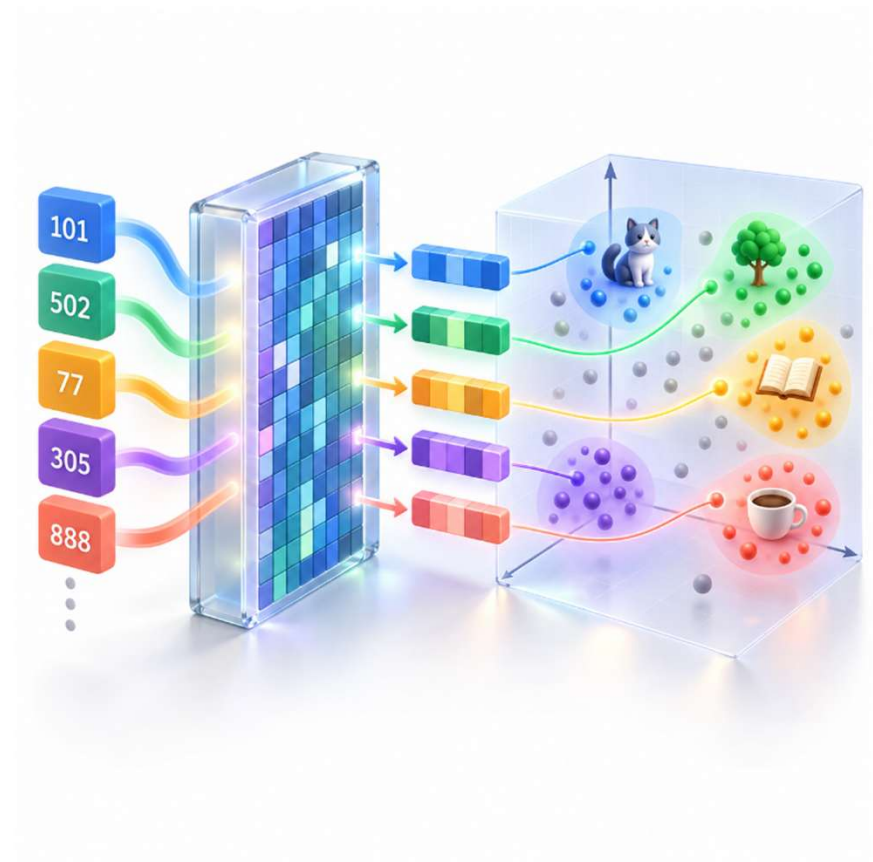


Token ID هنوز هیچ معنایی ندارد

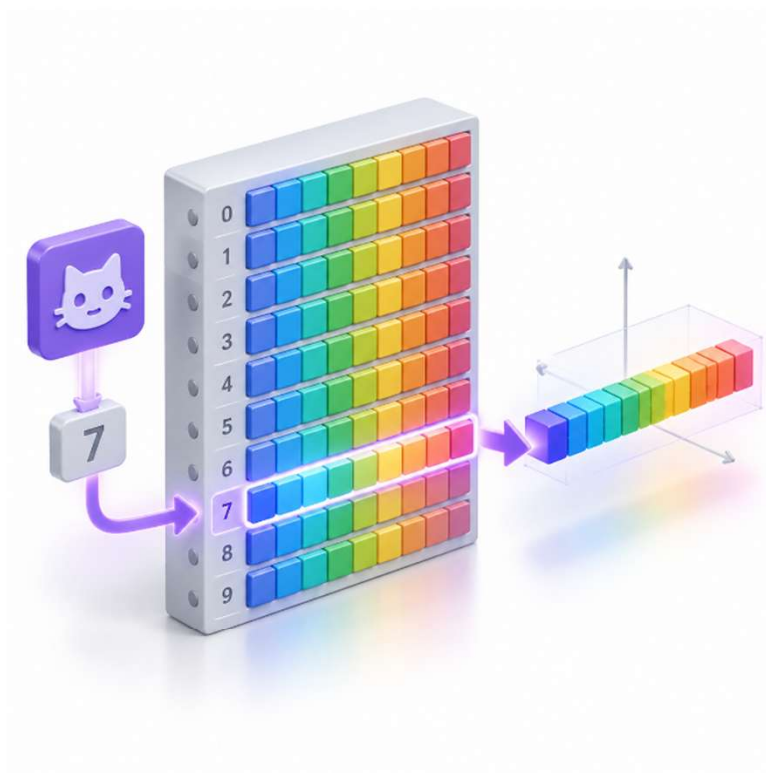
- هر Token یک شناسه عددی می‌گیرد
- Token ID فقط یک شماره است
- شماره بزرگ‌تر یعنی معنای بیشتر نیست
- مدل به Representation نیاز دارد



بخش ششم – از شناسه عددی تا معنا



Embedding؛ تبدیل شناسه به بازنمایی

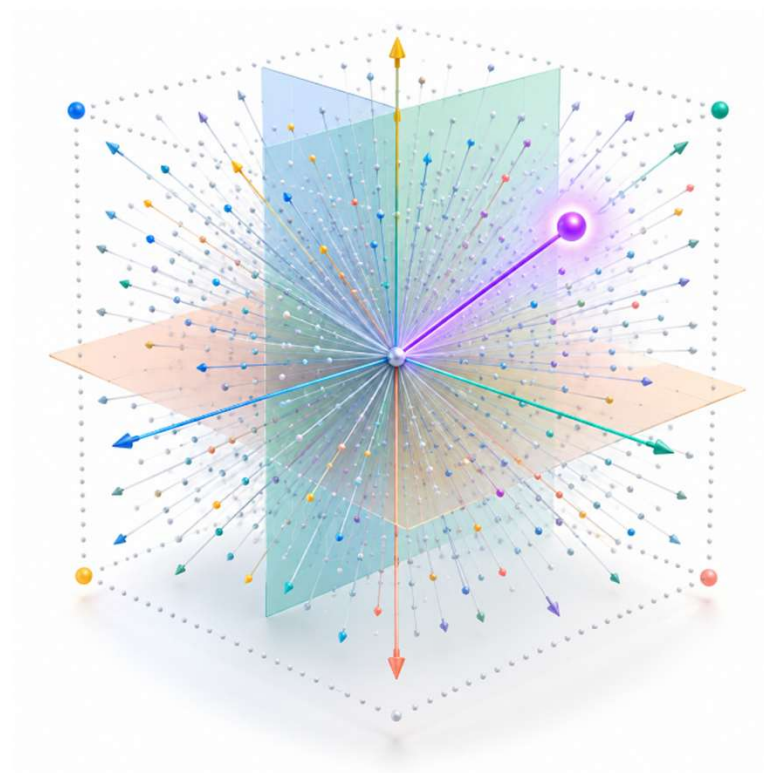


- Token ID وارد Embedding می‌شود
- خروجی یک Vector است
- Vector چندین Dimension دارد
- حالا مدل چیزی قابل محاسبه دارد



بُعد‌های Embedding چه معنایی دارند؟

- هر Token با مجموعه‌ای عدد توصیف می‌شود
- هیچ Dimension الزاماً معنای مستقل ندارد
- معنا میان چندین Dimension توزیع شده است
- Representation باید به‌صورت کلی دیده شود



Embedding Matrix هم یاد گرفته می‌شود

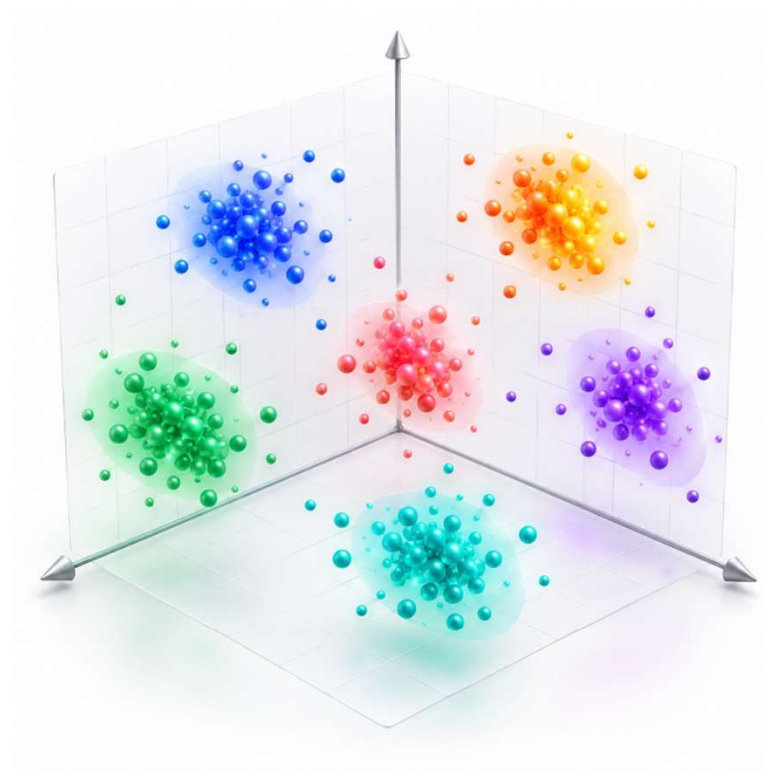


- Embedding از ابتدا معنای آماده ندارد
- مقادیر آن پارامترهای مدل هستند
- Gradient این مقادیر را تغییر می‌دهد
- Training بازنمایی مناسب را شکل می‌دهد



شباهت معنایی در فضا ظاهر می‌شود

- Representation های مشابه به هم نزدیک می‌شوند
- نزدیکی حاصل Training است
- فاصله می‌تواند شباهت را نشان دهد
- این پایه Semantic Search است



معنا در یک عدد پنهان نشده است



- Representation ماهیتی Distributed دارد
- یک مفهوم میان ابعاد مختلف پخش شده
- یک بُعد معمولاً معادل یک ویژگی نیست
- روابط از ترکیب ابعاد شکل می‌گیرند

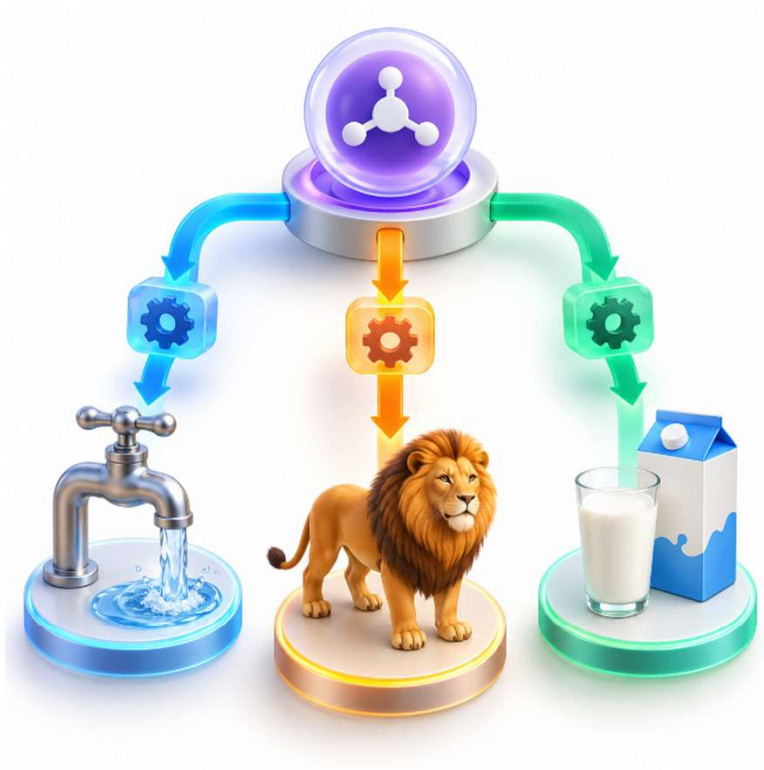


Embedding اولیه، پایان داستان نیست

- Token ابتدا Representation اولیه دارد
- Context هنوز وارد معنا نشده است
- لایه‌های مدل Representation را تغییر می‌دهند
- خروجی به Context وابسته می‌شود



یک «شیر»، چند بازنمایی متفاوت



- «شیر آب را ببند»
- «شیر در جنگل زندگی می‌کند»
- «شیر را داخل یخچال بگذار»
- Token مشابه، Context متفاوت
- Representation نهایی نیز متفاوت است





بخش هفتم —
این مفاهیم چه اثری بر
معماری دارند؟



Embedding داخلی و Embedding کاربردی یکی نیستند

- مدل مولد Representation داخلی می‌سازد
- Embedding Model هدف متفاوتی دارد
- Semantic Search معمولاً مدل خودش را می‌خواهد
- هر Vector برای هر کاربرد مناسب نیست



RAG آموزش دوباره مدل نیست



- دانش بیرونی هنگام اجرا بازیابی می‌شود
- پارامترهای مدل تغییر نمی‌کنند
- Retrieval اطلاعات مرتبط را پیدا می‌کند
- Source of Truth جای Model Knowledge نیست
- Semantic Search با Exact Match متفاوت است



از یادگیری مدل تا معماری سیستم

- Source of Truth $\neq$ Model Knowledge
- Business Decision $\neq$ Prediction
- Meaning $\neq$ Token ID
- Fact Database $\neq$ Embedding
- Training $\neq$ RAG
- Entire System $\neq$ AI Model

